TABLE OF CONTENTS

S. NO.	SECTION	PAGE
1	Title of Case Study	3
2	Abstract	3
3	Introduction and Real-Life Application	3
4	Literature Review	4
5	Objectives and Scope	4
6	C Programming Concepts Used	5
7	Methodology	6
8	System Design and Program Flow	7
9	How the C Programming Solves the Problem	8
10	Complete C Program Code	9–12
11	Output Screenshots and Test Cases	13
12	Limitations and Future Enhancements	14
13	Learning Outcomes and Conclusion	14
14	References	15

1. TITLE OF CASE STUDY

Smart Parking System

A menu-driven C application that manages ten parking slots for two-wheelers and four-wheelers, records vehicle entry and exit, calculates parking charges, and saves active parking information in a file.

Project type: C Programming simulation supported by DBMS concepts.

2. ABSTRACT

The Smart Parking System is a menu-driven application developed in the C programming language. It is designed to demonstrate how a parking facility can manage vehicle entry, parking-slot allocation, vehicle exit, parking charges and parking records in an organised way. The system supports ten parking slots and accepts both two-wheelers and four-wheelers.

The program uses a structure to represent each parking slot and an array to manage all ten slots. When a vehicle enters, the program checks for an available slot and assigns the first free slot. It stores the vehicle number, vehicle type and entry time. When the vehicle leaves, the program calculates the parking duration and charge. The selected rates are Rs. 20 per hour for a two-wheeler and Rs. 40 per hour for a four-wheeler. Any part of an hour is charged as a full hour.

File handling is used to save the parking-slot records in a local binary file so that active parking information can be loaded when the program is started again. The project also explains relevant DBMS ideas such as records, fields, primary keys, data consistency and persistent storage. The C application is an educational simulation; it does not connect to a real sensor network, payment gateway or SQL database.

3. INTRODUCTION AND DESCRIPTION OF REAL-LIFE APPLICATION

3.1 Introduction

A parking facility needs a reliable way to identify available spaces, record which vehicle is using each space and update the information when a vehicle leaves. In a traditional parking area, this work may be performed manually by a parking attendant. Manual recording can take time and may lead to incorrect slot allocation, misplaced records or confusion about the number of available spaces.

3.2 Description of the proposed application

This project develops a simplified console-based version of a Smart Parking System using C. It manages ten numbered parking slots. Both two-wheelers and four-wheelers can be registered. When a vehicle arrives, the program searches the slot array and allocates the first available slot. It then records the vehicle number, vehicle category and entry time.
When the vehicle exits, the user enters its vehicle number. The program searches the active records, calculates the elapsed time using the system clock and calculates the parking fee. The vehicle's slot is then marked as free. The program provides options to park a vehicle, remove a vehicle, display slot status, view active records and exit.

3.3 Real-life application

The same general idea can be used in shopping malls, hospitals, universities, office buildings, railway stations, airports and public parking areas. For example, a university parking facility can use a digital display to show available spaces near the entrance. A database-backed application can update the slot status after each entry or exit and allow an authorised operator to review parking records.

3.4 Role of DBMS in a real system

A Database Management System (DBMS) stores and manages related data. In a full Smart Parking System, tables could store parking slots, vehicles, parking sessions and payments. DBMS features such as keys, constraints, transactions and queries help maintain accurate information when many users access the system. In this classroom project, file handling is used for persistent storage; the program does not implement a full DBMS.

4.Literature Review

Parking management is a common example of an information system in which the availability of resources changes over time. Each parking space can be treated as a resource with a current state, such as available or occupied. The system must record when a vehicle starts using a slot and when that use ends.

A basic computerised parking application can use arrays and structures to represent slots and their details. Functions can divide the program into tasks such as vehicle registration, slot allocation, fee calculation and display of records. Conditional statements can validate input, while loops and switch-case statements can create a menu-driven interface.
In a larger information system, a DBMS can store the same information in related tables. A primary key uniquely identifies a record, while a foreign key connects related records. Constraints and transactions can help prevent invalid or conflicting updates. These concepts are relevant when multiple entry gates or operators update parking information at the same time.
The present project is a classroom-level simulation. It is based on the project requirements selected for this report and uses a local file to preserve active parking records. It does not claim to reproduce a commercial smart-parking product or a particular university's parking infrastructure.

5. OBJECTIVES AND SCOPE

5.1 Objectives

* To design a simple menu-driven Smart Parking System using C.
* To manage ten parking slots for two-wheelers and four-wheelers.
* To allocate the first available slot when a vehicle enters.
* To store a vehicle number, vehicle type and entry time for each occupied slot.
* To calculate parking charges at the selected hourly rates.
* To release a parking slot when the vehicle exits.
* To display available and occupied parking slots.

* To use structures, arrays, functions, loops, conditions and file handling.
* To introduce DBMS concepts such as records, fields, keys, consistency and persistent storage.

5.2 Scope

The project supports one local program instance and ten parking slots. It stores active parking information in a binary data file named parking_data.dat. It supports two vehicle categories and calculates the fee based on elapsed time. It is intended for learning and demonstration, not for handling real payments or controlling physical barriers.

5.3 Assumptions

* Each vehicle occupies one slot at a time.
* Vehicle numbers are entered as text, for example HR-XX-AB-1234.
* The first free slot is assigned automatically.
* Parking time is taken from the computer's system clock.
* Any started hour is charged as a complete hour, with a minimum charge of one hour.
* Parking rates are fixed in this version: Rs. 20/hour for two-wheelers and Rs. 40/hour for four-wheelers.

6. C PROGRAMMING CONCEPTS USED

CONCEPT	APPLICATION IN THE PROJECT
Structure (struct)	Groups slot number, vehicle number, vehicle type, occupied status and entry time into one record.
Structure variable	Each array element represents one parking slot.
User-defined functions	Separate functions handle loading, saving, parking, exit, display and fee calculation.
Pointers	File and array-related functions can receive addresses where required; the program uses standard C file pointers.
Array	Stores the details of all ten parking slots.
while loop	Keeps the menu running until the user chooses Exit.
time .h library	time() and difftime() are used to record entry time and calculate elapsed parking time.
File Handling	fopen(), fread(), fwrite() and fclose() load and save parking records.
switch-case	Selects the operation based on the menu choice.
if-else	Checks slot availability, vehicle type, duplicate vehicle numbers and valid menu choices.
Input/output	fgets() reads vehicle numbers; printf() displays menus and results.
Variables and data types	int stores slot numbers and menu choices; char arrays store vehicle details; time stores time values.

6.1 DBMS concepts explained

Database: An organised collection of related information. A full parking application could maintain a database of slots, vehicles and parking sessions.

Table: A collection of related records, such as a PARKING_SLOT table.

Record (row): Information about one slot or one parking session.

Field (column): A single attribute, such as slot number or vehicle number.

Primary key: A field that uniquely identifies a record. For example, slot_no can identify a parking slot.

Foreign key: A field that refers to a key in another table. For example, a parking-session record could refer to slot_no in PARKING_SLOT.

Data consistency: The system should not show one slot as both available and occupied at the same time.

Persistent storage: Information remains available after the program closes. This project demonstrates persistence using a file; a DBMS would provide database storage.

7. METHODOLOGY

The project is developed using a simple step-by-step method, moving from requirements to coding and testing.

1. Requirement identification: Identify the required operations: park a vehicle, remove a vehicle, view slot status, view active records and exit

2. Data design: Create a Parking Slot structure containing the slot number, vehicle number, vehicle type, occupied flag and entry time.

3. Array Creation: Create an array of ten Parking Slot elements to represent the ten parking spaces.

4. File Design: Use a local binary file named parking_data.dat to store the complete array of slot records.

5. Program modularisation: Create separate functions for initialising, loading, saving, parking, exiting, displaying status and calculating fees.

6. Vehicle entry: Validate the vehicle category and number, check for duplicates, find the first free slot, record the entry time and save the updated array.

7. Vehicle exit: Find the vehicle by number, calculate elapsed time and fee, display the result, clear the slot and save the updated array.

8. Compilation and testing: Compile the C program and test full parking, successful entry and exit, invalid vehicle type, duplicate vehicle number and file persistence.

8. SYSTEM DESIGN AND PROGRAM FLOW

Program workflow

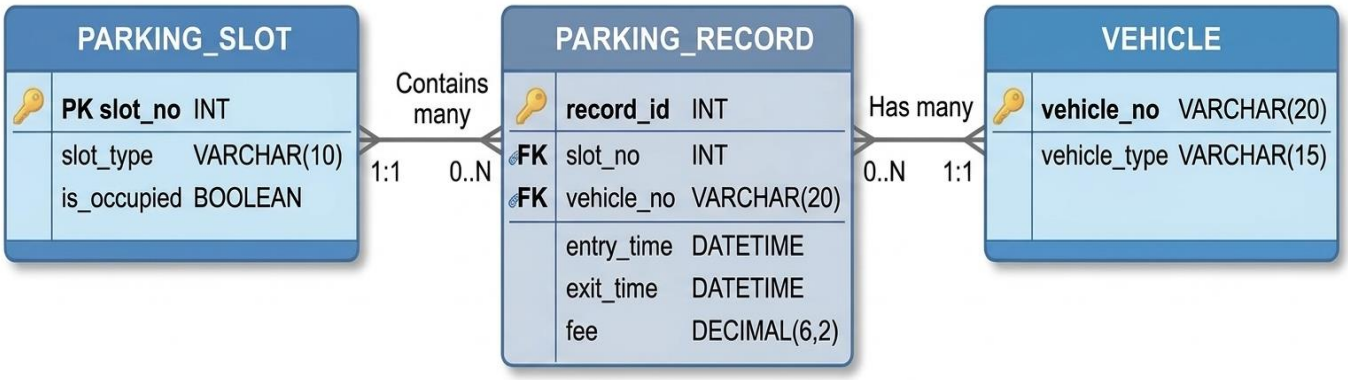
1. START
2. Load saved parking records from file
3. Display main menu
4. Read user's menu choice
5. Park vehicle Find free slot and save
6. Remove vehicle Calculate fee and save

7. View active records
8. Save records and EXIT
9. END

8.2 Data structure

```
typedef struct {
int slotNo;
char vehicleNo[20];
int vehicleType; /* 1 = two-wheeler, 2 = four-wheeler */
int isOccupied; /* 0 = free, 1 = occupied */
time_t entryTime;
} ParkingSlot;
ParkingSlot slots[10];
```

8.3 Conceptual database design



Conceptual design for a future database-backed version

The C program in this report stores the same basic information in a file, using structures and arrays; it does not connect to an SQL server.

The diagram represents a possible relational design for a future DBMS-based version. The C program in this report uses a structure array and a binary file instead of creating these SQL tables.

8.4 Module relationship

main() → loadData() → displayMenu() → parkVehicle() / removeVehicle() / displayStatus() / displayActiveRecords() → saveData()

9. HOW C PROGRAMMING SOLVES THE PROBLEM

SYSTEM OPERATION	PROGRAM ACTION
Vehicle Entry	The program checks whether the vehicle number is already parked, searches for a free slot and assigns the first available slot.
Slot availability	The is Occupied field indicates whether each slot is free or occupied.
Vehicle category	The vehicle Type field identifies a two-wheeler or four-wheeler.
Parking duration	The entry time is stored when the vehicle is parked. At exit, the current time is compared with the entry time.
Fee calculation	The program rounds any started hour up to a complete hour and multiplies it by the applicable rate.
Record persistence	The complete slot array is written to parking_data.dat after updates and loaded at the next program start.
Vehicle exit	The matching active record is found, the fee is displayed and the slot is cleared.

9.1 Parking charge calculation

Parking hours = ceiling of (elapsed seconds ÷ 3600), with a minimum of one hour.
Two-wheeler fee = parking hours × Rs. 20.
Four-wheeler fee = parking hours × Rs. 40.
Example: If a two-wheeler is parked for 1 hour and 10 minutes, the charged duration is 2 hours and the fee is Rs. 40. If a four-wheeler is parked for 2 hours and 5 minutes, the charged duration is 3 hours and the fee is Rs. 120.

9.2 How DBMS would solve the larger problem

In a real multi-user installation, a DBMS could store slot and session records centrally. A transaction could check that a slot is available and mark it occupied as one controlled operation. Constraints could prevent duplicate slot identifiers, and queries could retrieve available spaces. Concurrency control would be important if two entry gates attempted to allocate the same slot simultaneously.

10. COMPLETE C PROGRAM CODE

The following program is a beginner-friendly console simulation. It manages ten slots, supports two vehicle categories, calculates parking fees using the system clock and saves active records in a binary file. The file is created in the same folder from which the program is run.

Compile with a C compiler that supports the C11 standard. For example: gcc smart_parking.c -o smart_parking

```
/* SMART PARKING SYSTEM
C Programming project - 10 parking slots
Two-wheeler: Rs. 20 per hour
Four-wheeler: Rs. 40 per hour
Active records are saved in parking_data.dat
*/
#include <stdio.h>
#include <string.h>
#include <time.h>
#define TOTAL_SLOTS 10
#define DATA_FILE "parking_data.dat"
typedef struct {
int slotNo;
char vehicleNo[20];
int vehicleType; /* 1 = two-wheeler, 2 = four-wheeler */
int isOccupied; /* 0 = free, 1 = occupied */
time_t entryTime;
} ParkingSlot;
ParkingSlot slots[TOTAL_SLOTS];
/* Remove the newline added by fgets(). */
void removeNewline(char text[]) {
text[strcspn(text, "\n")] = '\0';
}
/* Read one line safely from the keyboard. */
void readLine(char text[], int size) {
if (fgets(text, size, stdin) != NULL) {
removeNewline(text);
} else {
text[0] = '\0';
}
}
/* Set up ten empty parking slots. */
void initialiseSlots(void) {
int i;
for (i = 0; i < TOTAL_SLOTS; i++) {
slots[i].slotNo = i + 1;
slots[i].vehicleNo[0] = '\0';
slots[i].vehicleType = 0;
slots[i].isOccupied = 0;
slots[i].entryTime = 0;
}
}
/* Load saved records. If no valid file exists, start empty. */
void loadData(void) {
FILE *fp = fopen(DATA_FILE, "rb");
if (fp == NULL) {
initialiseSlots();
return;
}
if (fread(slots, sizeof(ParkingSlot), TOTAL_SLOTS, fp)
!= TOTAL_SLOTS) {
initialiseSlots();
}
fclose(fp);
}
/* Save all ten slot records in the data file. */
void saveData(void) {
FILE *fp = fopen(DATA_FILE, "wb");
if (fp == NULL) {
printf("Error: Could not save parking records.\n");
return;
```

```
}
fwrite(slots, sizeof(ParkingSlot), TOTAL_SLOTS, fp);
fclose(fp);
}
/* Find a vehicle that is already parked. */
int findVehicle(const char vehicleNo[]) {
int i;
for (i = 0; i < TOTAL_SLOTS; i++) {
if (slots[i].isOccupied == 1 &&
strcmp(slots[i].vehicleNo, vehicleNo) == 0) {
return i;
}
}
return -1;
}
/* Display the current parking status. */
void displayStatus(void) {
int i, available = 0;
printf("\n------ PARKING SLOT STATUS ------\n");
for (i = 0; i < TOTAL_SLOTS; i++) {
printf("Slot %d: ", slots[i].slotNo);
if (slots[i].isOccupied == 1) {
printf("OCCUPIED - %s (%s)\n",
slots[i].vehicleNo,
slots[i].vehicleType == 1
? "Two-wheeler" : "Four-wheeler");
} else {
printf("AVAILABLE\n");
available++;
}
}
printf("Available slots: %d out of %d\n",
available, TOTAL_SLOTS);
}
/* Park a new vehicle in the first available slot. */
void parkVehicle(void) {
char vehicleNo[20];
char input[20];
int type, i;
printf("\nEnter vehicle number: ");
readLine(vehicleNo, sizeof(vehicleNo));
if (strlen(vehicleNo) == 0) {
printf("Vehicle number cannot be empty.\n");
return;
} printf("Select vehicle type:\n");
printf("1. Two-wheeler\n");
printf("2. Four-wheeler\n");
printf("Enter choice: ");
readLine(input, sizeof(input));
type = input[0] - '0';
if (type != 1 && type != 2) {
printf("Invalid vehicle type.\n");
return;
}
for (i = 0; i < TOTAL_SLOTS; i++) {
if (slots[i].isOccupied == 0) {
slots[i].isOccupied = 1;
strcpy(slots[i].vehicleNo, vehicleNo);
slots[i].vehicleType = type;
slots[i].entryTime = time(NULL);
saveData();
printf("\nVehicle parked successfully!\n");
printf("Allocated slot number: %d\n",
slots[i].slotNo);
return;
}
}
```

```
printf("Sorry, parking is full. No slot is available.\n");
}
/* Remove a vehicle and calculate its parking fee. */
void removeVehicle(void) {
char vehicleNo[20];
int index;
time_t exitTime;
double seconds;
long long hours;
int rate;
double fee;
printf("\nEnter vehicle number to exit: ");
readLine(vehicleNo, sizeof(vehicleNo));
index = findVehicle(vehicleNo);
if (index == -1) {
printf("Vehicle record not found.\n");
return;
}
exitTime = time(NULL);
seconds = difftime(exitTime, slots[index].entryTime);
/* Charge at least one hour; round a partial hour upward. */
hours = (long long)((seconds + 3599) / 3600);
if (hours < 1) {
hours = 1;
}
if (slots[index].vehicleType == 1) {
rate = 20;
} else {
rate = 40;
}
fee = hours * rate;
printf("\n---------- PARKING BILL ----------\n");
printf("Vehicle number : %s\n", slots[index].vehicleNo);
printf("Slot number : %d\n", slots[index].slotNo);
printf("Charged hours : %lld\n", hours);
printf("Rate per hour : Rs. %d\n", rate);
printf("Total fee : Rs. %.2f\n", fee);
printf("---------------------------------\n");
/* Mark the slot as free. */
slots[index].vehicleNo[0] = '\0';
slots[index].vehicleType = 0;
slots[index].isOccupied = 0;
slots[index].entryTime = 0;
saveData();
printf("Vehicle exited and slot released successfully.\n");
}
/* Display details of all currently parked vehicles. */
void displayActiveRecords(void) {
int i, found = 0;
printf("\n------ ACTIVE PARKING RECORDS ------\n");
for (i = 0; i < TOTAL_SLOTS; i++) {
if (slots[i].isOccupied == 1) {
printf("Slot: %d | Vehicle: %s | Type: %s\n",
slots[i].slotNo,
slots[i].vehicleNo,
slots[i].vehicleType == 1
? "Two-wheeler" : "Four-wheeler");
found = 1;
}
}
if (!found) {
printf("No vehicles are currently parked.\n");
}
}
int main(void) {
char input[20];
int choice;
loadData();
printf("================================\n");
printf(" SMART PARKING SYSTEM \n");
printf("================================\n");
do {
```

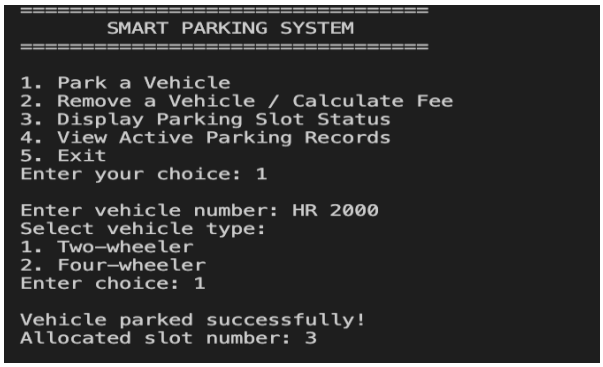
```
printf("\n1. Park a Vehicle\n");
printf("2. Remove a Vehicle / Calculate Fee\n");
printf("3. Display Parking Slot Status\n");
printf("4. View Active Parking Records\n");
printf("5. Exit\n");
printf("Enter your choice: ");
readLine(input, sizeof(input));
choice = input[0] - '0';
switch (choice) {
case 1:
parkVehicle();
break;
case 2:
removeVehicle();
break;
case 3:
displayStatus();
break;
case 4:
displayActiveRecords();
break;
case 5:
saveData();
printf("Records saved. Thank you!\n");
break;
default:
printf("Invalid choice. Please try again.\n");
}
} while (choice != 5);
return 0;
}
```

Important implementation note: The binary file stores C structure data for this educational program. Such files are suitable for a controlled classroom demonstration, but they are not a substitute for a portable, secure, multi-user database. Keep the program and its data file together in the same working folder.

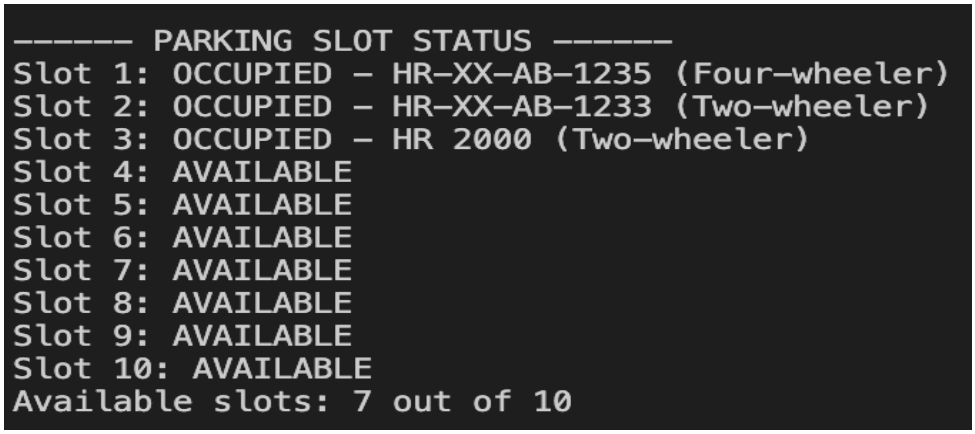
11. OUTPUT SCREENSHOTS AND TEST CASES

The following are sample console interactions and expected results for testing. They are illustrative examples; replace or supplement them with screenshots captured after compiling and running the program on your computer.

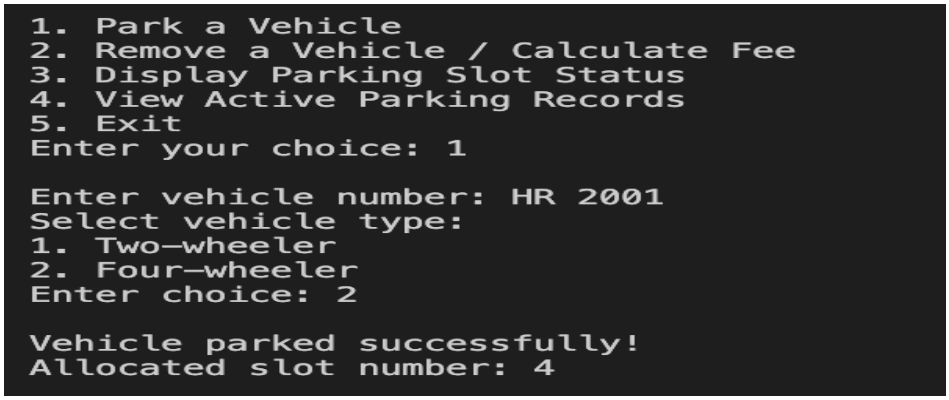
11.1 Output – Successful Vehicle Entry / Parking of 2-wheeler



11.2 Output – Parking status



11.3 Output – Four-wheeler Parking



11.4 Output - Enter the same vehicle number twice

```
1. Park a Vehicle
2. Remove a Vehicle / Calculate Fee
3. Display Parking Slot Status
4. View Active Parking Records
5. Exit
Enter your choice: 1

Enter vehicle number: HR 2000
This vehicle is already parked.
```

11.5 Output – Remove a vehicle / Calculate fee

```
1. Park a Vehicle
2. Remove a Vehicle / Calculate Fee
3. Display Parking Slot Status
4. View Active Parking Records
5. Exit
Enter your choice: 2

Enter vehicle number to exit: HR 2000

---------- PARKING BILL ----------
Vehicle number : HR 2000
Slot number    : 3
Charged hours  : 1
Rate per hour  : Rs. 20
Total fee      : Rs. 20.00
----------------------------------
Vehicle exited and slot released successfully.
```

11.6 Output – Exit Option to Save Records

```
1. Park a Vehicle
2. Remove a Vehicle / Calculate Fee
3. Display Parking Slot Status
4. View Active Parking Records
5. Exit
Enter your choice: 5
Records saved. Thank you!
```

12. LIMITATIONS AND FUTURE ENHANCEMENTS

12.1 Limitations

- The program supports only ten predefined parking slots.
- It is a single-computer console application and does not support simultaneous updates from multiple entry gates.
- The program does not use real parking sensors, cameras or automatic number-plate recognition.
- The program does not connect to a real SQL database or payment gateway.
- Parking charges are calculated using the computer's system clock; incorrect system time can affect the fee.
- The binary data file is intended for this compiled classroom program and may not be portable across different compilers or structure layouts.
- The program has basic input validation and is not designed as a secure commercial application.
- The program stores active parking records; it does not maintain a permanent transaction history after a vehicle exits.

12.2 Future enhancements

- Connect the application to a DBMS such as MySQL or SQLite.
- Create separate tables for parking slots, vehicles, parking sessions and payments.
- Add a graphical user interface or web/mobile application.
- Integrate sensors to detect vehicle presence automatically.
- Add an entry barrier and exit barrier integration.
- Maintain a permanent history of completed parking sessions and receipts.
- Add administrator login, user roles and stronger validation.
- Support multiple parking areas and different parking rates.
- Use database transactions and concurrency control to prevent two users from receiving the same slot.

13. LEARNING OUTCOMES AND CONCLUSION

13.1 Learning outcomes

• Understand how structures group related parking-slot information.
• Use arrays to manage a fixed collection of parking spaces.
• Create functions to divide a program into smaller tasks.
• Apply loops and switch-case statements to build a menu-driven application.
• Use conditional statements to validate vehicle details and slot availability.
• Use file handling to save and retrieve records between program runs.
• Use the time library to calculate elapsed parking duration.
• Understand how DBMS concepts such as tables, records, keys and data consistency apply to a real-life parking application.
• Recognise why a production system may need a central database, transactions and concurrency control.

13.2 Conclusion

The Smart Parking System demonstrates how C programming can be used to model a practical parking-management process. The application manages ten parking slots, accepts two-wheelers and four-wheelers, allocates available spaces, records entry time, calculates parking fees and releases slots when vehicles exit. Structures and arrays organise the data, functions divide the program into modules, and file handling preserves active parking records between program runs.

The project also connects the programming implementation with DBMS concepts. A real smart-parking application can use a database to manage slot information, vehicle records and parking sessions consistently, particularly when many users access the system. Although the present project is a simplified educational simulation and does not connect to a real database or hardware sensors, it provides a foundation for understanding how software and data management can improve a real-life parking service.

14. REFERENCES

• Classroom notes and practical material for C Programming.
• Classroom notes and learning material on Database Management Systems (DBMS).
• The C standard library documentation for stdio.h, string.h and time.h functions.
• Program design, sample test scenarios and outputs prepared for this Smart Parking System educational project.
• Note: The report describes a proposed educational simulation. No external commercial product documentation or research paper has been used to claim that this exact implementation is deployed in a real parking facility.

GitHub project link : https://github.com/Tushar-Rawal/C-Programming-Project-.git